

Università degli Studi di Salerno

Corso di Ingegneria del Software

Tasky Object Design Document Versione 0.5



Data: 19/12/2025

Progetto: Tasky	Versione: 0.5
Documento: Object Design Document	Data: 19/12/2025

Coordinatore del progetto:

Nome	Matricola
Luca Vincenzo Lambiase Fontana	0512113338

Partecipanti:

Nome	Matricola
Ilaria Lastra	0512120280
Rinaldo Perna	0512119593

Scritto da:	Ilaria Lastra
-------------	---------------

Revision History

Data	Versione	Descrizione	Autore
01/10/2025	0.1	Presentazione Proposta di Progetto	Lastra Ilaria
10/10/2025	0.2	Presentazione Problem Statement con descrizione degli scenari principali dell'applicazione, i suoi requisiti e gli ambienti target	Lastra Ilaria
20/10/2025	0.3	Presentazione del Requirements Analysis Document, con perfezionamento dei requisiti esposti nel Problem Statement e definizione dei criteri di successo e dello scopo e ambito del sistema	Lastra Ilaria
08/11/2025	0.3.1	Presentazione dell'Object Model e del Dynamic Model, con Class Diagram, Sequence Diagram e Statechart Diagram relativi al Requirements Analysis Document	Lastra Ilaria
19/11/2025	0.4	Presentazione del System Design Document, con discussione dei design goals, dell'architettura e mappatura hardware, software.	Lastra Ilaria, Luca Vincenzo Lambiase Fontana, Rinaldo Perna
19/12/2025	0.5	Presentazione dell'Object Design Document con discussione dei compromessi della progettazione ad oggetti, linee guida per la documentazione dell'interfaccia e packaging dei sottosistemi	Ilaria Lastra, Luca Vincenzo Lambiase Fontana, Rinaldo Perna

Sommario

1.	Introduzione.....	6
1.1.	Object design trade-offs	6
1.2.	Interface documentation guidelines	6
1.3.	Definitions, acronyms and abbreviations	6
1.4.	References	7
2.	Packages	7
3.	Class Interface Glossary	9

1. Introduzione

Il seguente documento è volto a presentare L'Object Design, il quale rappresenta la specifica dettagliata dell'architettura software a livello degli oggetti, definendo come i sottosistemi parte della nostra applicazione vengono realizzati concretamente attraverso classi, interfacce e interazioni. Qui rappresentiamo una guida definitiva per la fase di codifica (Implementation), garantendo che le decisioni architetturali siano tradotte in codice in modo coerente e manutenibile.

1.1. Object design trade-offs

I principali trade-off, ossia decisioni strategiche prese per risolvere i conflitti tra requisiti non funzionali contrastanti per la nostra applicazione Tasky sono:

- **Buy VS Build:** Abbiamo deciso di utilizzare componenti “off-the-shelf” consolidati per la gestione dei dati (MySQL) e per il framework web (Flask) concentrando lo sviluppo, ossia la Build, sulla logica di business specifica di Tasky (gestione dei progetti, task e dipendenti).
- **Gestione della Sessione (Stateless VS Stateful):** A differenza delle webapp tradizionali, TaskyAPI adotta un approccio Stateless. L'autenticazione avviene tramite Token. Questo elimina la necessità di mantenere sessioni in memoria sul server (HttpSession), rendendo l'API scalabile e facilmente consumabile da client diversi (App Android, Web Client).
- **Database Mapping:** Si è scelto di non utilizzare un ORM complesso (come SQLAlchemy ORM) ma di gestire le query SQL grezze direttamente tramite helper functions (repository.py) e driver PyMySQL. Questo garantisce il controllo totale sulle performance delle query e riduce l'overhead, mappando i risultati direttamente in dizionari Python o DTO (Marshmallow Schemas).

1.2. Interface documentation guidelines

La coerenza del codice è l'elemento che riduce principalmente la probabilità di errori durante l'integrazione. Per la realizzazione dell'implementazione della nostra applicazione, sono rigorosamente richieste delle specifiche convenzioni.

Per quanto riguarda gli standard di naming:

- **Classi:** I nomi delle classi vengono indicati usando sostantivi singolari e utilizzando nomenclatura PascalCase (es. DipendenteDAO)
- **Metodi:** Per i metodi utilizziamo una notazione camelCase (es. aggiungiTask)
- **Variabili:** Anche per le variabili utilizziamo una notazione camelCase
- **Costanti:** per le costanti utilizziamo UPPER_SNAKE_CASE (es. MAX_VALUE)
- **Package:** i nomi dei pacchetti dovranno essere in minuscolo e separati da punti (es. tasky.pacchetto)

Per la gestione degli errori e la robustezza del sistema:

- Gli errori non vengono gestiti tramite codici di ritorno, ma tramite il lancio di EccezioniCustom (es. ValidationException) che vengono catturate da handler centralizzati per restituire risposte JSON coerenti.
- L'input viene validato rigorosamente tramite Marshmallow Schemas prima di raggiungere la logica di business.

Infine, per la gestione delle collezioni all'interno del sistema, se un metodo deve restituire più oggetti, userà l'interfaccia List Collection piuttosto che array statici, per garantire flessibilità.

1.3. Definitions, acronyms and abbreviations

TERMINE	DEFINIZIONE
ORM	(Object-Relational Mapping) è una tecnica di programmazione (e spesso un framework software) che permette di far interagire un linguaggio di programmazione orientato agli oggetti (come Java, C# o Python) con un database relazionale (come MySQL o PostgreSQL).
DTO	(Data Transfer Object) è un design pattern il cui scopo principale è il raggruppamento di dati per ottimizzare il trasferimento tra diverse parti di un sistema, specialmente quando queste comunicano attraverso una rete.
PK	(Primary Key) Chiave Primaria, ossia un campo o un insieme di campi che identifica in modo univoco e inequivocabile ogni riga all'interno di una tabella
FK	(Foreign Key) Chiave Esterna, ossia lo strumento che permette di creare un legame logico tra due tabelle diverse.

1.4. References

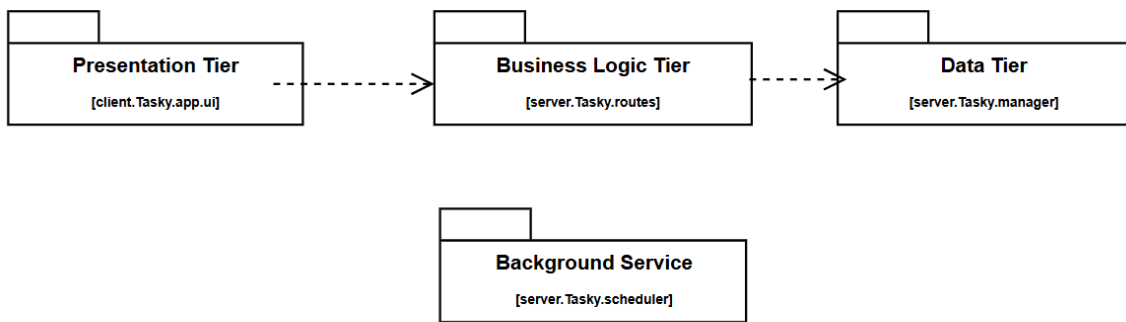
- Università degli Studi di Salerno - Corso di Ingegneria del Software - “Proposta di Progetto Tasky”
- Università degli Studi di Salerno - Corso di Ingegneria del Software - “Problem Statement”
- Università degli Studi di Salerno - Corso di Ingegneria del Software - “Requirements Analysis Document Tasky”
- Università degli Studi di Salerno - Corso di Ingegneria del Software - “System Design Document”

2. Packages

Il sistema Tasky è decomposto nei seguenti package, con lo scopo di massimizzare la coesione e minimizzare l'accoppiamento tra le componenti:

Pacchetti	Ruolo nel Design	Responsabilità principale
server.Tasky.routes	Controller	Espone gli endpoint REST (bd.route). Riceve JSON, chiama gli Schema per validare, delega a Repository e restituisce JSON http
server.Tasky.manager	Model	Esegue la logica di business dell'intero applicativo server
server.Tasky.schemas	Validator	Definisce gli schemi Marshmallow per validare i dati in ingresso (es. TaskCreateSchema) e serializzare quelli in uscita.
server.Tasky.scheduler	Background Service	Configura e gestisce i job periodici (APScheduler) per il controllo scadenze.
client.Tasky.app	Client	Applicazione Android per la visualizzazione dell'interfaccia UI per l'interazione con il server

2.1 Struttura dei pacchetti



2.2 Flow chart diagram del Sistema

2.2.1 Aggiunta Task



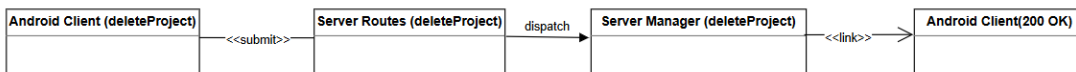
2.2.2 Eliminazione Task



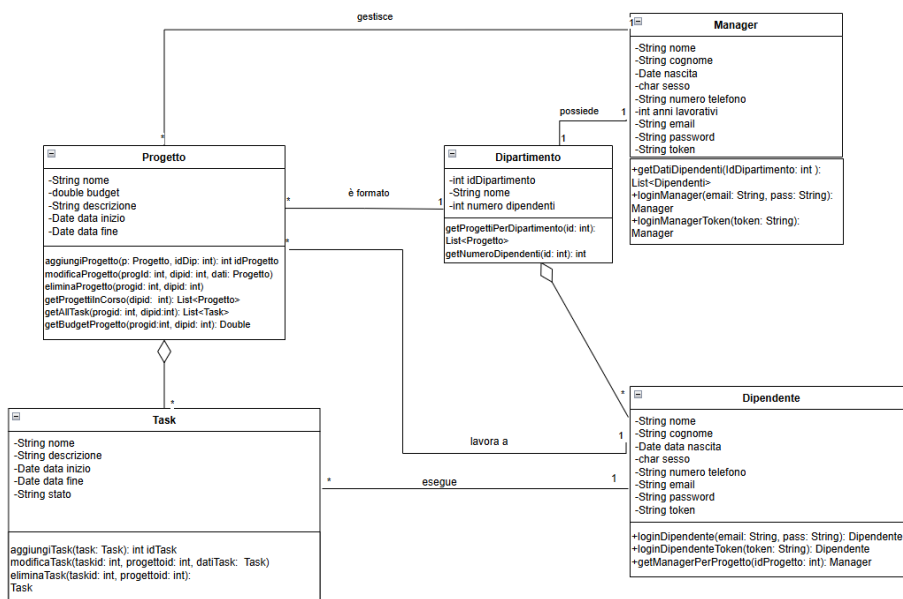
2.2.3 Aggiunta Progetto



2.2.4 Eliminazione Progetto



2.3 Class Diagram Ristrutturato



3. Class Interface Glossary

Qui di seguito vi è la descrizione delle principali classi presenti all'interno del nostro sistema.

Pacchetto: server.Tasky.manager

Classi	Attributi	Metodi	Descrizione
Dipendente	- email (String PK) - password (String, hashed) - nome (String) - cognome (String) - data_nascita (Date) - sex (String) - numero_telefono (String) - id_department (int, FK)	+loginDipendente (String email, String pass) +loginDipendenteToken (String token)	Rappresenta un utente generico del sistema
Manager	- anni_lavorativi (Int) - idDepartment(int) - email (String PK) - password (String, hashed) - nome (String) - cognome (String) - data_nascita (Date) - sex (String)	+loginManager (String email, String pass) +loginManagerToken (String token) +handleDipendentiDataByDepartment(int idDepartment)	Manager estende dipendente con privilegi amministrativi, aggiungendo logica relativa alla possibilità di creare progetti e assegnare task
Progetto	- id_progetto (Int, PK) - nome (String) - descrizione (String) - budget (Float) - data_inizio (Date) - data_fine (Date) - id_department (Int, FK) - emailManager(String)	+getProjectsFromDepartment(int idDepartment) +getTasksFromProject(int idProgetto) +insertProject(Progetto pogetto, int idDepartment) +deleteProject(int idProgetto) +getProjectsForManager(String emailManager) Getter	Rappresenta un progetto aziendale
Task	- id (Int, PK) - nome (String) - descrizione (String) - stato (String: 'Nuovo', 'In Corso', 'Completato') - data_inizio (Date) - data_fine (Date) - id_progetto (Int, FK) - emailDipendente (String, FK) - emailManager(String)	+getInProgressTasksForEmployee(String emailDipendente) +getCompletedTasksForEmployee(String emailDipendente) +getSuspendedTasksForEmployee(String emailDipendente) +handleUpdateTaskStatus(Task task) +handleUpdateTask(Task task) +deleteTask(int id) +insertTask(Task task, int idProgetto, String emailDipendente, String emailManager) Getter	Unità di lavoro assegnata a un dipendente
Dipartimento	- id (int, PK) - nome (String) - numero dipendenti (int)	Getter +handleDipendentiByDepartment(int	Rappresenta un dipartimento aziendale, al quale appartengono un

		idDipartimento)	manager e dei dipendenti
--	--	-----------------	--------------------------

Pacchetto: server.Tasky.routes

Classi	Attributi	Metodi	Descrizione
Authentication	- token (String) - email (String) - password (String)	+login(String email, String password or String token?)	Classe con la quale viene gestita l'autenticazione utente
TaskController	- dati task - token (String) - emailDipendente (String) - idDipartimento (int)	+tasksInProgress(String token, String emailDipendente) +tasksCompleted(String token, String emailDipendente) +tasksSuspended(String token, String emailDipendente) +updateTaskStatus (int id, String nuovoStato) +updateTask(int id, Task task) +deleteTask(int id) +addTask(Task task)	Classe tramite la quale avviene la gestione delle funzionalità dei tasks e permette il collegamento tra l'applicazione client e il server back-end
ProjectController	- dati progetto - id_dipartimento (int) - emailManager (String)	+projectByDepartment(String token, int id_dipartimento) +taskByProject(String token, int id_progetto, int id_dipartimento) +addProject(Project project) +deleteProject(int idProgetto) +projectsByManager(String emailManager)	Classe con la quale avviene la gestione delle funzionalità dei progetti, permettendone il collegamento con il server back-end
DipendenteController	id_dipartimento (int)	+dipendentiDataByDepartment(String token, int id_dipartimento)	Classe con la quale avviene la gestione dei dipendenti di uno specifico dipartimento, collegando l'applicazione client e il server back-end

Pacchetto: client.Tasky.app

Classi	Attributi	Metodi	Descrizione
DipendenteDTO	- nome (String) - cognome (String) - dataNascita (Date) - email (String) - num_telefono (int) - sex (String) - id_Dipartimento (int)	Getter e setter	Rappresentazione DTO del Dipendente
ManagerDTO	- anni_lavorativi (int) - attributi dipendente	Getter e setter	Rappresentazione DTO del Manager

ProgettoDTO	• id_progetto (int) • nome (String) • descrizione (String) • budget (Float) • data_inizio (Date) • data_fine (Date) • id_dipartimento (int)	Getter e setter	Rappresentazione DTO del Progetto
TaskDTO	• id (int) • nome (String) • descrizione (String) • stato (String) • data_inizio (Date) • data_fine (Date) • id_progetto (int) • email_dipendente (String) • email_manager (String)	Getter e setter	Rappresentazione DTO del Task
TaskAdder	• id_dipartimento (int) • id_progetto (int) • nome (String) • stato (String) • descrizione (String) • data_inizio (Date) • data_fine (Date) • email_dipendente (String) • email_manager (String) • token (String)	+onCreate(savedInstanceState: Bundle?) +taskAdder(dati task, String token)	Classe tramite la quale viene visualizzata l'activity relativa al modulo di aggiunta di una task
ProjectAdder	• token (String) • nome (String) • descrizione (String) • budget (float) • data_inizio (Date) • data_fine (Date) • id_dipartimento (int)	+projectAdder (String token, dati progetto) +onCreate(savedInstanceState: Bundle?)	Classe tramite la quale viene visualizzata l'activity relativa al modulo di aggiunta di un progetto
HomeDipendente	• sesso • id_dipartimento • nome_dipartimento • nome • email	+homeDipendenteActivity(dati dipendente) +onCreate(savedInstanceState: Bundle?)	Classe tramite la quale viene visualizzata la pagina principale relativa alle informazioni e funzionalità del dipendente
HomeManager	• sesso • id_dipartimento • nome_dipartimento • nome • email	+homeManagerActivity(dati manager) +onCreate(savedInstanceState: Bundle?)	Classe tramite la quale viene visualizzata la pagina principale relativa alle informazioni e funzionalità del manager
InfoViewerTask	• token (String) • id (int) • nome (String) • stato (String) • descrizione (String) • data_inizio (Date) • data_fine (Date) • email_dipendente (String) • email_manager (String)	+infoTask(String token, int id, dati task) +onCreate(savedInstanceState: Bundle?)	Classe tramite la quale viene visualizzata la pagina relativa alle informazioni del task indicato
InfoViewerProject	• id (int) • nome (String)	+infoProject(dati progetto)	Classe tramite la quale viene visualizzata la pagina relativa

	• budget (float) • descrizione (String) • data_inizio (Date) • data_fine (Date)	+onCreate(savedInstanceState: Bundle?)	alle informazioni del progetto indicato
ListTaskViewerByDipendente	• token (String) • emailDipendente (String) • Tasks (List<Task>)	+onCreate(savedInstanceState: Bundle?) +listTaskViewer(String token, String emailDipendente)	Classe tramite la quale viene visualizzata la lista dei tasks appartenenti ad un dipendente specificato
ListTaskViewerByProject	• token (String) • idProgetto (int) • idDipartimento (int)	+onCreate(savedInstanceState: Bundle?) +listTaskViewer(String token, int idProgetto, int idDipartimento)	Classe tramite la quale viene visualizzata la lista di tasks relativi ad un singolo progetto
ListProjectViewer	• token (String) • id_dipartimento (int) • Progetti (List<Progetti>)	+onCreate(savedInstanceState: Bundle?) +listProjectViewer(String token, int id_dipartimento)	Classe tramite la quale viene visualizzata la lista dei progetti appartenenti al dipartimento di appartenenza
ListDipendenteViewer	• token (String) • id_dipartimento (int) • Dipendenti (List<Dipendente>)	+onCreate(savedInstanceState: Bundle?) +listDipendenteViewer(String token, int idDipartimento)	Classe tramite la quale viene visualizzata la lista dei dipendenti appartenenti al dipartimento di appartenenza
Login	• email (String) • password (String)	+onCreate(savedInstanceState: Bundle?) +login(String email, String password)	Classe tramite la quale viene visualizzata la pagina per l'accesso all'applicazione
TaskSospender	• token (String) • id_task (int) • nome (String) • stato (String) • descrizione (String) • data_inizio (Date) • data_fine (Date) • email_dipendente (String) • email_manager (String)	+onCreate(savedInstanceState: Bundle?) +taskSospender(String token, dati task)	Classe tramite la quale viene visualizzata la pagina relativa al modulo per la sospensione di un task indicato

Pacchetto: server.Taskv.schemas

Classi	Attributi	Metodi	Descrizione
TaskSchema	• dati task	+taskUpdateSchema(Schema) +taskAddSchema(Schema) +taskDeleteSchema(Schema)	Classe tramite la quale vengono validati i dati passati in input, riguardanti i task, ai controller
ProjectSchema	• dati progetto	+projectAddSchema(Schema) +projectDeleteSchema(Schema) +projectUpdateSchema(Schema)	Classe tramite la quale vengono validati i dati passati in input, riguardanti i progetti, ai controller
LoginSchema	• email • password • token	+loginSchema(Schema)	Classe tramite la quale vengono validati i dati passati in input, riguardanti il login, ai controller

ManagerSchema	• dati manager	+managerProjectSchema(Schema)	Classe tramite la quale vengono validati i dati passati in input, riguardanti il manager, ai controller
---------------	--	-------------------------------	---

Pacchetto: server.Tasky.scheduler

Classi	Attributi	Metodi	Descrizione
Initializer		+initScheduler()	Classe che permette l'inizializzazione e start del background scheduler
TimeTrigger		+checkOverdueTasks()	Classe che garantisce il controllo sulla scadenza dei tasks e se scaduti, setta il loro stato a "Sospeso". Viene eseguita ogni giorno alle 00:00

3.1 Descrizione dei metodi e dei loro vincoli

In questa sezione descriveremo ed illustreremo al meglio le operazioni associate alle singole classi descritte, indicando i possibili vincoli. Quest'ultimi saranno espressi in OCL (Object Constraint Language).

Pacchetto: server.Tasky.manager

- Class: Dipendente

Invarianti:

context Dipendente inv emailValida: self.email.includes('@')

context Dipendente inv emailFormato: self.email.matches('^([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,})\$')

- Class: Manager

Invarianti:

context Manager inv managerAssegnato: not self.Dipartimento_id_dipartimento.isUndefined()

- Class: Progetto

Invarianti:

context Progetto inv coerenzaDateProgetto: self.dataInizio <= self.dataFine

context Progetto inv budgetNonNegativo: self.budgetIstanziato >= 0

context Progetto inv formatoData: self.data_inizio.format = '%d/%m/%Y'

- Class: Task

Invarianti:

context Task inv coerenzaDataTask: self.data_inizio <= self.data_fine

inv statiAmmessi: Set { 'IN_PROGRESS', 'DONE', 'SUSPENDED' } → includes(self.stato)

context Task inv taskNelRangeProgetto: self.data_inizio >= self.Progetto.dataInizio and
self.data_fine <= self.progetto.dataFine

context Task inv coerenzaDipartimentoAssegnazione: self.Dipendente.Dipartimento_id_dip =
Self.Progetto.Dipartimento_id_dip

Pacchetto: server.Tasky.routes

- Class: TaskController

Invarianti:

context TaskController
inv ReferentialIntegrity:
Project.allInstances()->exists(p | p.id = self.id_progetto) and
Dipendente.allInstances()->exists(d | d.email = self.email_dipendente) and
Manager.allInstances()->exists(m | m.email = self.email_manager)

- **Class: ProjectController**

Invarianti:

context ProjectController
inv DepartmentIdMatch:
self.Dipartimento_id_dipartimento = self.dipartimento.id
inv IsManager: self.oclIsTypeOf(Manager)

- **Class: DipendenteController**

Invarianti:

context DipendenteController
inv DepartmentIdMatch:
self.Dipartimento_id_dipartimento = self.dipartimento.id

- **Class: Autentication**

Invarianti:

context Autentication inv EmailNotNull: email <> null and email <> ''
inv PasswordNotNull: self.password <> null and password <> ''

Pacchetto: server.Tasky.schemas

- **Class: TaskSchema**

Invarianti:

context TaskSchema
inv ValidStatus:
Set {'IN_PROGRESS', 'DONE', 'SUSPENDED'}->includes(self.stato)
context TaskSchema inv TokenNotNull : self.token <> null

- **Class: ProjectSchema**

Invarianti:

context ProjectSchema inv ChronologicalDates: self.data_inizio <= self.data_fine
inv PositiveBudget: self.budget >= 0
context ProjectSchema inv TokenNotNull : self.token <> null

- **Class: LoginSchema**

Invarianti:

context LoginSchema inv CredentialsRequirement: (self.token.notEmpty()) xor (self.email.notEmpty() and
self.password.notEmpty())
inv ValidTokenMapping: self.isValid implies User.allInstances()->exists(u | u.token = self.token)

Pacchetto: server.Tasky.scheduler

Non vi sono vincoli per le classi appartenenti al seguente pacchetto

Pacchetto: client.Tasky.app

- **Class: TaskAdder**

Invarianti:

context TaskAdder inv TokenNotNull : self.token <> null
inv IsManager: self.oclIsTypeOf(Manager)

Nome metodo	taskAdder
Descrizione	Questo metodo permette al manager di poter inserire un

	nuovo task
Precondizioni	context TaskAdder :: taskAdder(t: Task) pre TaskNotNull: t <> null
PostCondizioni	/

- **Class: ProjectAdder**

Invarianti:

context TaskAdder inv TokenNotNull : self.token <> null
 inv IsManager: self.oclIsTypeOf(Manager)

Nome metodo	projectAdder
Descrizione	Questo metodo permette ad un manager di inserire un nuovo progetto
Precondizioni	context ProjectAdder :: projectAdder(p : Project) pre ProjectNotNull: p <> null
PostCondizioni	/

- **Class: HomeDipendente**

Invarianti:

context HomeDipendente inv DipendenteNotNull : self.email <> null

- **Class: HomeManager**

Invarianti:

context HomeManager inv ManagerNotNull: self.email <> null

- **Class: InfoViewerTask**

Invarianti:

context InfoViewerTask inv TaskNotNull : self.id <> null

- **Class: InfoViewerProject**

Invarianti:

context InfoViewerProject inv ProjectNotNull: self.id <> null

Nome metodo	infoProject
Descrizione	Questo metodo permette ad un manager di poter visualizzare le informazioni relative ad un progetto
Precondizioni	context InfoViewerProject :: infoProject(p : Project) pre ManagerNotNull: self.emailManager <> null
PostCondizioni	/

- **Class: ListTaskViewerByDipendente**

Invarianti:

context ListTaskViewerByDipendente inv TokenNotNull: self.token <> null
 inv EmailNotNull: self.emailDipendente <> null

- **Class: ListTaskViewerByProject**

Invarianti:

context ListTaskViewerByProject inv TokenMustBeValid: (not token.oclIsUndefined()) implies token.isValid()
 inv TokenBelongsToManager: self.user.oclIsTypeOf(Manager)
 inv ProjectNotNull: self.idProgetto <> null
 inv DepartmentNotNull: self.idDipartimento <> null

- **Class: ListProjectViewer**

Invarianti:

Context ListProjectViewer inv TokenNotNull: self.token <> null
 inv IsManager: self.oclIsTypeOf(Manager)
 inv DepartmentNotNull: self.idDipartimento <> null

- **Class: ListDipendenteViewer**

Invarianti:

Context ListDipendenteViewer inv TokenNotNull: self.token <> null
 inv IsManager: self.ocIsTypeOf(Manager)
 inv DepartmentNotNull: self.idDipartimento <> null

- **Class: Login**

Invarianti:

context Login inv EmailNotNull: email <> null and email <> ''
 inv PasswordNotNull: self.password <> null and password <> ''

Nome metodo	login
Descrizione	Questo metodo permette ad un utente di accedere all'applicazione
Precondizioni	context Login: login(e: Email, p: Password) pre TokenNotNull: (not token.ocIsUndefined()) implies token.isValid()
PostCondizioni	/

- **Class: TaskSospender**

Invarianti:

Context TaskSospender inv TaskNotNull: self.id <> null
 inv IsManager: self.ocIsTypeOf(Manager)

4. Glossary

TERMINE	DEFINIZIONE
Tasky	Applicazione Task Manager aziendale
API RESTful	Un'interfaccia di programmazione applicativa (API) che aderisce ai principi di progettazione dell'architettura REST (Representational State Transfer) .
Tasky API RESTful	Interfaccia RESTful esposta dal nostro server di backend realizzata tramite il framework Flask
Three – tier architecture	L'architettura Three-Tier è l'evoluzione del modello Client-Server ed è lo standard per le applicazioni enterprise e web moderne. Suddivide l'applicazione in tre strati logici (e spesso fisici) distinti: Presentation Tier (Livello di Presentazione), Application Tier (Livello Logico o Business Logic) e Data Tier (Livello Dati)
Schema Marshmallow	Classe responsabile della validazione e serializzazione/deserializzazione dei dati (DTO).
Token	Stringa crittografata utilizzata per identificare l'utente senza sessione server-side.
Entity	Rappresenta l' informazione persistente e a lungo termine gestita dal sistema.
Token	Stringa crittografata utilizzata per identificare l'utente senza sessione server-side.
Trade – offs	Una decisione strategica in cui si accetta di sacrificare o ridurre una determinata qualità o risorsa del sistema per ottenerne un vantaggio in un'altra.